

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC PHENIKAA



BÁO CÁO DỰ ÁN CUỐI KÌ
CHỦ ĐỀ 14: HỆ THỐNG QUẢN LÝ ĐẶT XE ĐƯA ĐÓN SÂN
BAY VÀ ĐIỂM DU LỊCH

Giảng viên : Ths. Phạm Đức Bắc

Học phần : Lập trình web ứng dụng trong lịch 2-1-3-25(N01)

Khóa : K17

Họ và tên sinh viên :

Trần Quang Vy MSSV : 23015177

Bùi Thị Mai Hoa MSSV : 23015809

Hà Nội, tháng 5 năm 2026

Mục lục

MỞ ĐẦU	5
1 CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG	6
1.1 Bài toán thực tế và Nhu cầu hệ thống	6
1.1.1 Phát biểu bài toán	6
1.1.2 Nhu cầu giải quyết phần mềm	6
1.2 Mục tiêu và Phạm vi thiết kế	7
1.2.1 Mục tiêu hệ thống	7
1.2.2 Phạm vi thiết kế	7
1.3 Cơ sở lý thuyết về Kiến trúc và Bảo mật	8
1.3.1 Kiến trúc Model-View-Controller (MVC)	8
1.3.2 Mô hình kiểm soát truy cập (RBAC)	8
1.4 Làm rõ công nghệ sử dụng và Hiện thực hóa mã nguồn	9
1.4.1 Framework PHP Laravel và Trình đóng gói Eloquent ORM	9
1.4.2 Kiến trúc Frontend tối giản với Tailwind CSS và Alpine.js	9
2 PHÂN TÍCH HỆ THỐNG	10
2.1 Biểu đồ Use case và Đặc tả Tác nhân	10
2.1.1 Xác định và phân loại Tác nhân (Actors)	10
2.1.2 Thiết kế sơ đồ Use Case tổng thể	10
2.1.3 Đặc tả chi tiết các Use Case trọng tâm	12
2.2 Phân tích và Thiết kế Phân hệ Chức năng	12
2.2.1 Phân hệ Xác thực và Định danh (Authentication Module)	12
2.2.2 Phân hệ Khách hàng (User Web Interface)	13
2.2.3 Phân hệ Quản trị viên (Admin Command Control)	13
2.3 Thiết kế Cơ sở dữ liệu lô-gíc (Database Schema)	13
2.3.1 Định nghĩa cấu trúc chi tiết các bảng dữ liệu	14
Bảng users	14
Bảng vehicles	14
Bảng drivers	15
Bảng transfer_routes	15
Bảng bookings	15
2.3.2 Mô hình Quan hệ Thực thể liên kết (Entity-Relationship Diagram)	16
2.4 Sơ đồ Luồng Xử lý Nghiệp vụ Hệ thống (Request Lifecycle)	16
3 THIẾT KẾ CHI TIẾT VÀ HIỆN THỰC HÓA MÃ NGUỒN	16
3.1 Hiện thực hóa kiến trúc Laravel MVC trong dự án	16
3.1.1 Thành phần Model và Cơ chế ánh xạ quan hệ dữ liệu	18
3.1.2 Thành phần Controller và Phân tách không gian xử lý nghiệp vụ	18
3.1.3 Thành phần View và Cơ chế xây dựng linh kiện giao diện nâng cao	19
3.2 Hiện thực hóa các Module chức năng cốt lõi	20
3.2.1 Module Xác thực hệ thống và Quản lý phân quyền nâng cao	20
3.2.2 Module Quản lý Tuyến xe và Đặt vé chuyến đi Khách hàng	20
3.2.3 Module Quản lý tài nguyên Đội xe và Phê duyệt đơn hàng phía Admin	21
3.3 Hiện thực hóa Trung tâm phân tích dữ liệu Bảng điều khiển (Dashboard Engine)	22

4 KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ	23
4.1 Môi trường triển khai thực nghiệm	23
4.1.1 Cấu hình hạ tầng phần cứng thực nghiệm	23
4.1.2 Hệ sinh thái phần mềm và Phiên bản công nghệ lõi	23
4.2 Kịch bản kiểm thử hệ thống (Test Cases Execution)	23
4.2.1 Kịch bản Kiểm thử Module Xác thực và Màng lọc phân quyền Admin	23
4.2.2 Kịch bản Kiểm thử Nghiệp vụ Tính toán Tổng giá đơn Đặt vé phía Khách hàng	24
4.3 Demo giao diện trực quan của Hệ thống (UI Screenshots)	25
4.3.1 Phân hệ Khách hàng (User Interface Landing and History)	25
4.3.2 Phân hệ Không gian Điều hành của Quản trị viên (Admin Panel Dashboards)	26
4.4 Đánh giá ưu điểm và hạn chế của sản phẩm phần mềm	26
4.4.1 Ưu điểm công nghệ vượt trội	26
4.4.2 Các điểm hạn chế cần cải thiện	26
4.5 Bảng phân công và đánh giá mức độ hoàn thành công việc	27
KẾT LUẬN	28
1. Kết quả đạt được của Đề tài	28
2. Định hướng và Kiến nghị phát triển trong tương lai	28
TÀI LIỆU THAM KHẢO	29

Danh sách hình vẽ

2.1	Biểu đồ Use Case tổng thể phân rã các phân hệ chức năng	11
2.2	Sơ đồ mối quan hệ giữa các thực thể vật lý dưới cơ sở dữ liệu (ERD)	17
2.3	Sơ đồ luồng tuần tự xử lý yêu cầu nghiệp vụ phía Server	18
4.1	Giao diện lưới danh sách các tuyến xe đưa đón công khai dành cho hành khách	25
4.2	Giao diện biểu mẫu đặt vé an toàn và bảng danh mục theo dõi lịch trình lịch sử cá nhân	25
4.3	Giao diện trung tâm phân tích dữ liệu kinh tế và theo dõi trạng thái hệ thống theo thời gian thực . .	26

Danh sách bảng

2.1	Cấu trúc chi tiết bảng dữ liệu users	14
2.2	Cấu trúc chi tiết bảng dữ liệu vehicles	14
2.3	Cấu trúc chi tiết bảng dữ liệu drivers	15
2.4	Cấu trúc chi tiết bảng dữ liệu transfer_routes	15
2.5	Cấu trúc chi tiết bảng dữ liệu bookings	15
4.1	Kịch bản kiểm thử phân hệ Xác thực và An ninh hệ thống	24
4.2	Kịch bản kiểm thử luồng nghiệp vụ đặt vé chuyển xe	24
4.3	Bảng phân công nhiệm vụ nhân sự	27

MỞ ĐẦU

Sự phát triển mạnh mẽ của ngành du lịch và nhu cầu đi lại bằng đường hàng không ngày càng tăng cao đã kéo theo sự nở rộ của các dịch vụ vận tải hành khách kết nối sân bay và các trung tâm đô thị. Tuy nhiên, qua khảo sát thực tế tại nhiều đơn vị vận tải, quy trình quản lý xe đưa đón sân bay hiện nay phần lớn vẫn diễn ra thủ công. Việc lưu trữ dữ liệu bằng sổ sách, Excel hay điều phối qua tin nhắn, cuộc gọi dẫn đến nhiều hệ lụy như: thất lạc thông tin đơn đặt xe, trùng lặp lịch trình của tài xế, và đặc biệt là tốn rất nhiều thời gian, nhân lực để tính toán doanh thu.

Nhận thấy những hạn chế đó, cùng với mong muốn áp dụng các kiến thức đã học về kỹ nghệ phần mềm vào thực tiễn, nhóm chúng em quyết định chọn đề tài: **"Phân tích, thiết kế và xây dựng hệ thống quản lý dịch vụ đưa đón sân bay dựa trên kiến trúc MVC"**.

Mục tiêu của đề tài:

- **Về mặt lý thuyết:** Nghiên cứu và ứng dụng framework Laravel (PHP), kiến trúc MVC, và các công nghệ giao diện hiện đại (Tailwind CSS, Alpine.js) vào việc phát triển ứng dụng Web.
- **Về mặt thực tiễn:** Xây dựng thành công một hệ thống phần mềm hoàn chỉnh cho phép hành khách dễ dàng tra cứu, đặt xe trực tuyến; đồng thời cung cấp cho Quản trị viên (Admin) một không gian điều hành tập trung để quản lý tuyến đường, xe, tài xế và thống kê doanh thu theo thời gian thực.

Bố cục của báo cáo: Ngoài phần Mở đầu và Kết luận, báo cáo được chia thành 4 chương chính:

- **Chương 1: Cơ sở lý thuyết và công nghệ sử dụng:** Trình bày bối cảnh bài toán và các nền tảng công nghệ (Laravel, MySQL, Tailwind) được sử dụng để xây dựng hệ thống.
- **Chương 2: Phân tích và thiết kế hệ thống:** Phân rã các chức năng qua biểu đồ Use Case, sơ đồ cơ sở dữ liệu (ERD) và luồng xử lý nghiệp vụ.
- **Chương 3: Cấu trúc mã nguồn và Phát triển ứng dụng:** Đi sâu vào kiến trúc thư mục, cách cấu hình Route, Controller, Model và kỹ thuật phân quyền.
- **Chương 4: Kết quả thực nghiệm và Đánh giá:** Trình bày các giao diện đã hoàn thiện, kịch bản kiểm thử và bảng đánh giá phân công nhiệm vụ của nhóm.

Chương 1: CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG

1.1 Bài toán thực tế và Nhu cầu hệ thống

1.1.1 Phát biểu bài toán

Trong bối cảnh nền kinh tế toàn cầu hóa và xu hướng số hóa mạnh mẽ hiện nay, nhu cầu di chuyển, vận tải hành khách nói chung và dịch vụ đưa đón hành khách tại các cửa ngõ hàng không lớn (như sân bay) nói riêng đang tăng trưởng với tốc độ phi mã. Sân bay không chỉ là đầu mối giao thông cốt lõi mà còn là bộ mặt hạ tầng của một quốc gia, nơi tiếp đón hàng triệu lượt khách du lịch nội địa và quốc tế mỗi năm. Điều này đặt ra một thách thức lớn đối với việc tổ chức, điều phối các phương thức vận tải kết nối giữa sân bay và các trung tâm đô thị một cách nhịp nhàng, tối ưu và an toàn.

Tuy nhiên, qua khảo sát thực tế tại các đơn vị vận tải vừa và nhỏ, quy trình quản lý và vận hành dịch vụ xe đưa đón sân bay hiện tại vẫn mang tính chất thủ công, thô sơ và phân tán. Các doanh nghiệp thường duy trì phương thức quản lý truyền thống dựa trên các công cụ văn phòng cơ bản như Microsoft Excel, sổ chép tay, hoặc thông qua hệ thống tổng đài điện thoại kết nối trực tiếp với tài xế. Cách tiếp cận này bộc lộ những lỗ hổng và điểm nghẽn nghiêm trọng trong quy trình vận hành phần mềm và kỹ nghệ hệ thống (Software Engineering):

- **Sự phân tán và mất đồng bộ dữ liệu (Data Isolation):** Toàn bộ thông tin về danh mục phương tiện, hồ sơ nhân sự của tài xế, dữ liệu tuyến đường và thông tin hành khách được lưu trữ riêng rẽ dưới dạng tệp tin cục bộ hoặc ghi chép rời rạc. Khi một khách hàng thực hiện yêu cầu đặt chuyến, nhân viên điều hành phải đối chiếu thủ công giữa lịch trình xe chạy, trạng thái sẵn sàng của tài xế và số ghế trống còn lại của phương tiện. Quy trình này tốn nhiều thời gian và dễ xảy ra sai sót thông tin (Human Errors).
- **Rủi ro xung đột lịch trình và lãng phí nguồn lực:** Việc thiếu hụt một hệ thống kiểm soát trạng thái theo thời gian thực (Real-time State Management) dẫn đến hiện tượng điều phối trùng lặp, một tài xế hoặc một phương tiện bị xếp lịch cho hai chuyến đi chồng chéo thời gian. Ngược lại, có những khung giờ phương tiện trống lịch nhưng không được khai thác tối đa, gây lãng phí nghiêm trọng về chi phí cơ hội và khấu hao tài sản vận tải.
- **Khó khăn trong việc kiểm soát doanh thu và dữ liệu khách hàng:** Do thông tin giao dịch tài chính thu thập qua nhiều kênh (tiền mặt, chuyển khoản, khách đặt qua tổng đài), ban quản lý rất khó tổng hợp báo cáo doanh thu theo ngày, tuần hoặc tháng một cách chính xác. Việc theo dõi lịch sử đơn đặt vé của từng khách hàng để phục vụ cho các chiến lược chăm sóc, giữ chân khách hàng (CRM) gần như là bất khả thi.

Từ những bất cập thực tế trên, việc xây dựng một hệ thống phần mềm tập trung hóa, tự động hóa quy trình đặt vé và quản trị vận tải là một yêu cầu vô cùng cấp thiết nhằm nâng cao năng lực cạnh tranh và chuẩn hóa quy trình kỹ nghệ của doanh nghiệp.

1.1.2 Nhu cầu giải quyết phần mềm

Để giải quyết triệt để các bài toán cốt lõi đã phát biểu, hệ thống phần mềm được thiết kế và xây dựng bắt buộc phải đáp ứng toàn diện ba trục nhu cầu kỹ thuật cốt lõi sau đây:

1. **Trục Quản lý Tài nguyên và Ràng buộc Dữ liệu Bền vững (Relational Data Integrity):** Hệ thống cần tổ chức và chuẩn hóa cơ sở dữ liệu quan hệ đạt các tiêu chuẩn 3NF. Việc thiết lập cấu trúc bảng phải phản ánh chính xác các ràng buộc vật lý trong thực tế: Một phương tiện phải có biển số xe duy nhất, số ghế cố định; một tài xế phải được định danh qua số giấy phép lái xe; và một tuyến đường vận chuyển phải được cấu hình chặt chẽ với một phương tiện và một tài xế cụ thể nhằm tối ưu hóa luồng vận hành, loại bỏ hoàn toàn tình trạng trùng lặp dữ liệu tĩnh.

2. **Trực Tự động hóa Nghiệp vụ và Đồng bộ hóa Trạng thái (Automated Business Logic):** Hệ thống phải cung cấp một cơ chế tính toán tự động các tham số kinh tế. Khi khách hàng lựa chọn tuyến đường và nhập số lượng hành khách đi cùng, phần mềm sẽ tự động truy vấn giá vé gốc của tuyến đường đó từ cơ sở dữ liệu, thực hiện phép toán nhân để tính toán ra tổng chi phí giao dịch (Total Price) ngay tại tầng logic của ứng dụng trước khi ghi nhận vào cơ sở dữ liệu. Luồng dữ liệu này phải chuyển trạng thái một cách tuần tự (từ chờ duyệt sang đã xác nhận hoặc đã hủy) dưới sự kiểm duyệt trực quan của ban điều hành.
3. **Trực Bảo mật Hệ thống và Phân quyền Nghiêm ngặt (Access Control Framework):** Phần mềm cần thiết lập một rào cản bảo mật vững chắc thông qua các bộ lọc yêu cầu HTTP (Middleware). Việc phân chia vai trò (RBAC) giữa người dùng phổ thông (User) và quản trị viên (Admin) phải được kiểm soát tuyệt đối ở mức mã nguồn tầng xử lý (Server-side validation). Khách hàng chỉ được quyền tương tác với các tài nguyên cá nhân như tra cứu tuyến xe đang hoạt động, xem lịch sử đặt vé của chính mình. Trong khi đó, các quyền hạn mang tính chất thay đổi cấu trúc hệ thống hoặc can thiệp vào tài nguyên dùng chung (Thêm/Sửa/Xóa phương tiện, tài xế, phê duyệt đơn hàng) phải được cô lập hoàn toàn cho tài khoản Admin, chặn đứng mọi nguy cơ khai thác lỗ hổng leo thang đặc quyền từ phía Client.

1.2 Mục tiêu và Phạm vi thiết kế

1.2.1 Mục tiêu hệ thống

Mục tiêu tổng quát của đề tài nghiên cứu này là ứng dụng quy trình kỹ nghệ phần mềm hiện đại để phân tích, thiết kế và hiện thực hóa một ứng dụng Web quản lý dịch vụ đưa đón sân bay (Airport Transfer) toàn diện. Hệ thống hướng tới việc đạt được các mục tiêu kỹ thuật cụ thể sau đây:

- **Về mặt kiến trúc và mã nguồn:** Triển khai thành công mẫu thiết kế Model-View-Controller (MVC) chuẩn công nghiệp thông qua khung làm việc Laravel. Đảm bảo cấu trúc mã nguồn tường minh, sạch sẽ, tách biệt hoàn toàn giữa logic tương tác cơ sở dữ liệu, logic điều hướng nghiệp vụ và tầng hiển thị giao diện để nâng cao khả năng bảo trì và mở rộng hệ thống trong tương lai.
- **Về mặt quản trị nghiệp vụ:** Cung cấp cho ban điều hành doanh nghiệp một không gian làm việc tập trung (Admin Panel) trực quan. Tự động hóa 100% việc thu thập và tính toán dữ liệu thống kê, hiển thị tổng số lượng đơn đặt vé, số đơn đang chờ xử lý, số đơn đã hủy và tổng doanh thu thực tế theo thời gian thực dưới dạng biểu đồ trực quan để hỗ trợ ra quyết định kinh doanh.
- **Về mặt trải nghiệm người dùng (UX/UI):** Xây dựng giao diện phản hồi linh hoạt (Responsive Design), tương thích hoàn hảo trên cả máy tính để bàn (Desktop) và thiết bị di động (Mobile). Tối ưu hóa thời gian tải trang và phản hồi biểu mẫu (Form Response) xuống dưới 1.5 giây, tích hợp các bộ lọc tìm kiếm điểm đón, điểm trả thông minh và cơ chế cảnh báo lỗi nhập liệu trực tiếp tại Client để mang lại trải nghiệm mượt mà nhất cho khách hàng.

1.2.2 Phạm vi thiết kế

Để đảm bảo tính khả thi, độ tập trung chuyên sâu và hoàn thiện sản phẩm phần mềm chất lượng cao trong khuôn khổ bài tập lớn, dự án xác định rõ phạm vi thiết kế và triển khai giới hạn như sau:

- **Về hạ tầng kiến trúc:** Dự án tập trung phát triển ứng dụng Web dựa trên mô hình kết xuất phía máy chủ (Server-Side Rendering) kết hợp với các công cụ tối giản Front-end. Dự án không mở rộng quy mô sang hạ tầng kiến trúc hướng dịch vụ (Microservices) hoặc triển khai các giải pháp cân bằng tải phức tạp (Load Balancing) nhằm giữ cho hệ thống gọn nhẹ, tập trung tối đa vào hiệu năng lõi và tính toàn vẹn dữ liệu.
- **Về phân hệ người dùng:** Hệ thống giới hạn tập trung giải quyết triệt để quy trình đặt vé của Khách hàng đã được xác thực (Authenticated Users) thông qua luồng đăng ký/đăng nhập chuẩn và phân hệ điều hành tối cao

của Quản trị viên (Admin). Hệ thống chưa tích hợp phân hệ dành riêng cho tài xế (Driver App cục bộ) hoặc cơ chế định vị toàn cầu GPS theo thời gian thực của xe di chuyển trên bản đồ, thông tin tài xế và phương tiện hiện tại được điều phối cố định theo cấu hình tuyến xe của Admin.

- **Về giải pháp thanh toán:** Hệ thống tập trung xử lý luồng logic nghiệp vụ của đơn đặt vé và tính toán tổng số tiền giao dịch trên môi trường kiểm thử phần mềm. Việc thanh toán được giả định thực hiện qua phương thức thanh toán trực tiếp hoặc chuyển khoản sau khi đơn hàng chuyển sang trạng thái "confirmed", hệ thống chưa kết nối trực tiếp với các cổng thanh toán trung gian trực tuyến của bên thứ ba (như VNPay, MoMo, hay Stripe).

1.3 Cơ sở lý thuyết về Kiến trúc và Bảo mật

1.3.1 Kiến trúc Model-View-Controller (MVC)

Mẫu kiến trúc Model-View-Controller (MVC) là một trong những nền tảng kinh điển và phổ biến nhất trong công nghệ phần mềm hiện đại, đóng vai trò là kiến trúc cốt lõi xuyên suốt quá trình xây dựng hệ thống Airport Transfer này. Triết lý cốt lõi của MVC là nguyên lý "Tách biệt các mối quan tâm" (Separation of Concerns - SoC), chia một ứng dụng phần mềm thành ba thành phần biệt lập có chức năng chuyên biệt:

1. **Model (Thành phần Mô hình Dữ liệu):** Đảm nhận vai trò quản lý toàn bộ trạng thái dữ liệu và logic nghiệp vụ cốt lõi của ứng dụng. Model trực tiếp giao tiếp với hệ quản trị cơ sở dữ liệu quan hệ, thực thi các truy vấn dữ liệu, kiểm tra các ràng buộc thực thể và thực hiện các tính toán logic. Model hoàn toàn độc lập với giao diện hiển thị; nó không quan tâm dữ liệu sẽ được hiển thị như thế nào trên trình duyệt của người dùng. Trong hệ thống, các lớp như `Booking`, `TransferRoute`, `Vehicle` chính là các Model đại diện cho các bảng dữ liệu vật lý dưới DB.
2. **View (Thành phần Hiển thị Giao diện):** Chịu trách nhiệm biên dịch dữ liệu nhận được từ Model thành cấu trúc giao diện trực quan hiển thị cho người dùng cuối (thường là HTML, CSS, và JavaScript). View không chứa các logic xử lý nghiệp vụ phức tạp mà chỉ chứa các cú pháp lặp, rẽ nhánh cơ bản để render dữ liệu động vào mã nguồn tĩnh. Toàn bộ các tệp Blade Template trong dự án đóng vai trò là tầng View của kiến trúc này.
3. **Controller (Thành phần Bộ điều khiển Logic):** Đóng vai trò là cầu nối điều phối, điều khiển luồng tương tác giữa người dùng, Model và View. Khi người dùng gửi một yêu cầu (HTTP Request) từ trình duyệt, Controller sẽ tiếp nhận, phân tích tham số, gọi các phương thức xử lý tương ứng trong Model để thay đổi dữ liệu, sau đó đóng gói dữ liệu kết quả và đẩy sang tầng View để kết xuất giao diện trả về cho Client (HTTP Response).

Sự cô lập và phân tầng nghiêm ngặt này mang lại những ưu thế công nghệ vượt trội: Giúp mã nguồn có tính module hóa cao, dễ dàng thực hiện kiểm thử đơn vị (Unit Testing) trên tầng logic; khi thay đổi cấu trúc giao diện ở View, logic nghiệp vụ ở Controller và Model hoàn toàn được giữ nguyên; đồng thời giảm thiểu tối đa xung đột mã nguồn (Code Conflicts) khi triển khai phát triển dự án theo nhóm.

1.3.2 Mô hình kiểm soát truy cập (RBAC)

Kiểm soát truy cập dựa trên vai trò (Role-Based Access Control - RBAC) là mô hình bảo mật chuẩn công nghiệp được ứng dụng để giải quyết bài toán phân quyền và quản lý tài nguyên trong hệ thống Airport Transfer. Nguyên lý cốt lõi của RBAC là quyền truy cập vào các tài nguyên phần mềm không được gán trực tiếp cho từng tài khoản người dùng cụ thể, mà được gán gián tiếp thông qua các "Vai trò" (Roles) trung gian được định nghĩa trước trong hệ thống.

Trong kiến trúc cơ sở dữ liệu của dự án, thuộc tính `role` được triển khai dưới dạng kiểu dữ liệu liệt kê (Enum) trong bảng `users`, giới hạn nghiêm ngặt với hai giá trị thực thể: `'user'` và `'admin'`. Logic vận hành của mô hình RBAC trong hệ thống được hiện thực hóa qua quy trình xử lý luồng nghiêm ngặt sau:

- **Định danh và Duy trì Phiên làm việc (Authentication & Session):** Khi người dùng gửi thông tin đăng nhập hợp lệ, hệ thống sẽ khởi tạo một phiên làm việc (Session ID) được lưu trữ an toàn phía Server và đính kèm Cookie định danh tương ứng ở trình duyệt Client. Phiên làm việc này bọc toàn bộ thông tin thực thể của người dùng hiện tại bao gồm khóa chính id, email, và giá trị role.
- **Kiểm soát và Đánh chặn tại Tầng Lọc (Middleware Isolation):** Đối với mọi HTTP Request hướng tới các tài nguyên nhạy cảm (như đường dẫn quản trị /admin/*), hệ thống áp dụng cơ chế đánh chặn chủ động thông qua lớp Middleware bảo mật. Middleware này sẽ trích xuất thông tin người dùng từ Session, thực hiện đối sánh điều kiện logic logic để quyết định cho phép Request tiếp tục đi sâu vào hệ thống hoặc ngay lập tức hủy luồng yêu cầu, trả về mã trạng thái lỗi HTTP 403 Forbidden.

Giải pháp này loại bỏ hoàn toàn các rủi ro bảo mật liên quan đến việc rò rỉ dữ liệu hoặc lỗi logic IDOR (Insecure Direct Object Reference) nguy hiểm, đảm bảo tính toàn vẹn hệ thống ở mức tối đa.

1.4 Làm rõ công nghệ sử dụng và Hiện thực hóa mã nguồn

1.4.1 Framework PHP Laravel và Trình đóng gói Eloquent ORM

Hệ thống quản lý dịch vụ đưa đón sân bay được xây dựng trên nền tảng Back-end phía Server mạnh mẽ là Framework PHP Laravel phiên bản 12 mới nhất, kết hợp với môi trường PHP 8.2 trở lên. Laravel được lựa chọn nhờ vào hệ sinh thái công cụ toàn diện và cơ chế bảo mật mặc định (Secure by Default) vô cùng nghiêm ngặt, giúp lập trình viên tập trung tối đa vào thiết kế logic nghiệp vụ thay vì phải xử lý các vấn đề hạ tầng thô sơ.

- **Cơ chế bảo vệ an toàn dữ liệu tự động:** Laravel tích hợp sẵn các màng lọc an ninh chống lại ba loại lỗ hổng bảo mật phổ biến nhất trên môi trường ứng dụng web. Lỗ hổng giả mạo yêu cầu chéo trang (CSRF) được chặn đứng thông qua việc ép buộc đính kèm một mã hóa độc quyền (CSRF Token) vào mọi Request dạng POST/PUT/PATCH. Lỗ hổng chèn mã lệnh script độc hại (XSS) được vô hiệu hóa hoàn toàn nhờ cơ chế escape dữ liệu mặc định của Blade Engine thông qua cú pháp `{{ }}`. Lỗ hổng chèn mã độc SQL (SQL Injection) được loại bỏ tận gốc nhờ kỹ thuật liên kết tham số (Parameter Binding) của tầng cơ sở dữ liệu.
- **Bản chất xử lý ánh xạ đối tượng Eloquent ORM:** Điểm nhấn công nghệ cốt lõi của Laravel là trình đóng gói Eloquent ORM (Object-Relational Mapping). Eloquent trừu tượng hóa các bảng dữ liệu vật lý trong MySQL thành các lớp lập trình hướng đối tượng độc lập. Thay vì phải viết các câu lệnh SQL thuần phức tạp với các phép nối JOIN cồng kềnh dễ gây nghẽn tài nguyên hệ thống, Eloquent cho phép định nghĩa các mối quan hệ thực thể ngay trong mã nguồn PHP thông qua các hàm liên kết như `hasMany()` và `belongsTo()`.

Hệ thống tận dụng triệt để chất xám thiết kế của Eloquent để thiết lập mối quan hệ giữa các thực thể một cách mượt mà: Một tuyến đường (TransferRoute) sở hữu nhiều lượt đặt chỗ thông qua hàm `bookings()`, và một đơn đặt chỗ (Booking) sẽ ánh xạ ngược lại một cách chính xác tới một tài khoản người dùng duy nhất và một tuyến đường duy nhất, giúp tối ưu hóa hiệu năng truy vấn dữ liệu xuống DB.

1.4.2 Kiến trúc Frontend tối giản với Tailwind CSS và Alpine.js

Để tối ưu hóa giao diện hiển thị thích ứng (Responsive UI) trên đa thiết bị và tăng tốc độ tải trang xuống mức tối thiểu, hệ thống ứng dụng mô hình phát triển Frontend hiện đại dựa trên sự kết hợp hoàn hảo giữa Tailwind CSS và thư viện JavaScript Alpine.js, loại bỏ hoàn toàn gánh nặng tải của các bộ khung Single Page Application cồng kềnh như React hay Vue.

- **Tailwind CSS và cơ chế biên dịch Just-In-Time (JIT):** Khác với các bộ CSS dựng sẵn truyền thống chứa hàng ngàn class dư thừa, Tailwind CSS hoạt động theo trường phái Utility-first. Kết hợp với trình đóng gói tài nguyên siêu tốc **Vite** cấu hình trong tệp `tailwind.config.js`, trình biên dịch JIT sẽ quét toàn bộ các tệp tin Blade View (`*.blade.php`) của dự án theo thời gian thực. Hệ thống chỉ giữ lại và biên dịch những

class thực sự được sử dụng trong giao diện để xuất ra một tệp tin CSS tĩnh duy nhất có dung lượng cực kỳ tối giản (chỉ vài chục KB). Điều này giúp trình duyệt của Client giảm thiểu số lượng Request tải tài nguyên, tối ưu hóa tốc độ kết xuất đồ họa màn hình.

- **Alpine.js và Cơ chế Phản hồi DOM nội tại (Declarative DOM):** Đối với các tương tác động phía Client cần sự phản hồi tức thì (như đóng mở cửa sổ hộp thoại Modal khi xác nhận xóa tài xế, ẩn hiện thanh menu điều hướng Navigation, thả xuống Dropdown menu tài khoản cá nhân), hệ thống sử dụng Alpine.js. Thư viện này chèn trực tiếp các thuộc tính phản xạ tự nhiên (Reactive Attributes như `x-data`, `x-show`, `x-on:click`) thẳng vào cây phân cấp DOM hiện tại của Blade View. Cách tiếp cận này giúp xử lý các tương tác UI mượt mà ngay tại trình duyệt của người dùng mà không cần tiêu tốn tài nguyên hệ thống để khởi chạy một Virtual DOM phức tạp, giữ cho ứng dụng luôn đạt độ nhạy bén cao nhất về mặt trải nghiệm thị giác.

Chương 2: PHÂN TÍCH HỆ THỐNG

2.1 Biểu đồ Use case và Đặc tả Tác nhân

Trong quy trình phát triển và kỹ nghệ phần mềm, việc xác định rõ ràng ranh giới hệ thống, các tác nhân tương tác và các ca sử dụng (Use case) là bước đi tiên quyết giúp định hình toàn bộ kiến trúc logic của ứng dụng. Hệ thống Quản lý Dịch vụ Đưa đón Sân bay (Airport Transfer System) được xây dựng dựa trên sự phân tách nghiêm ngặt quyền hạn truy cập của các thực thể sử dụng.

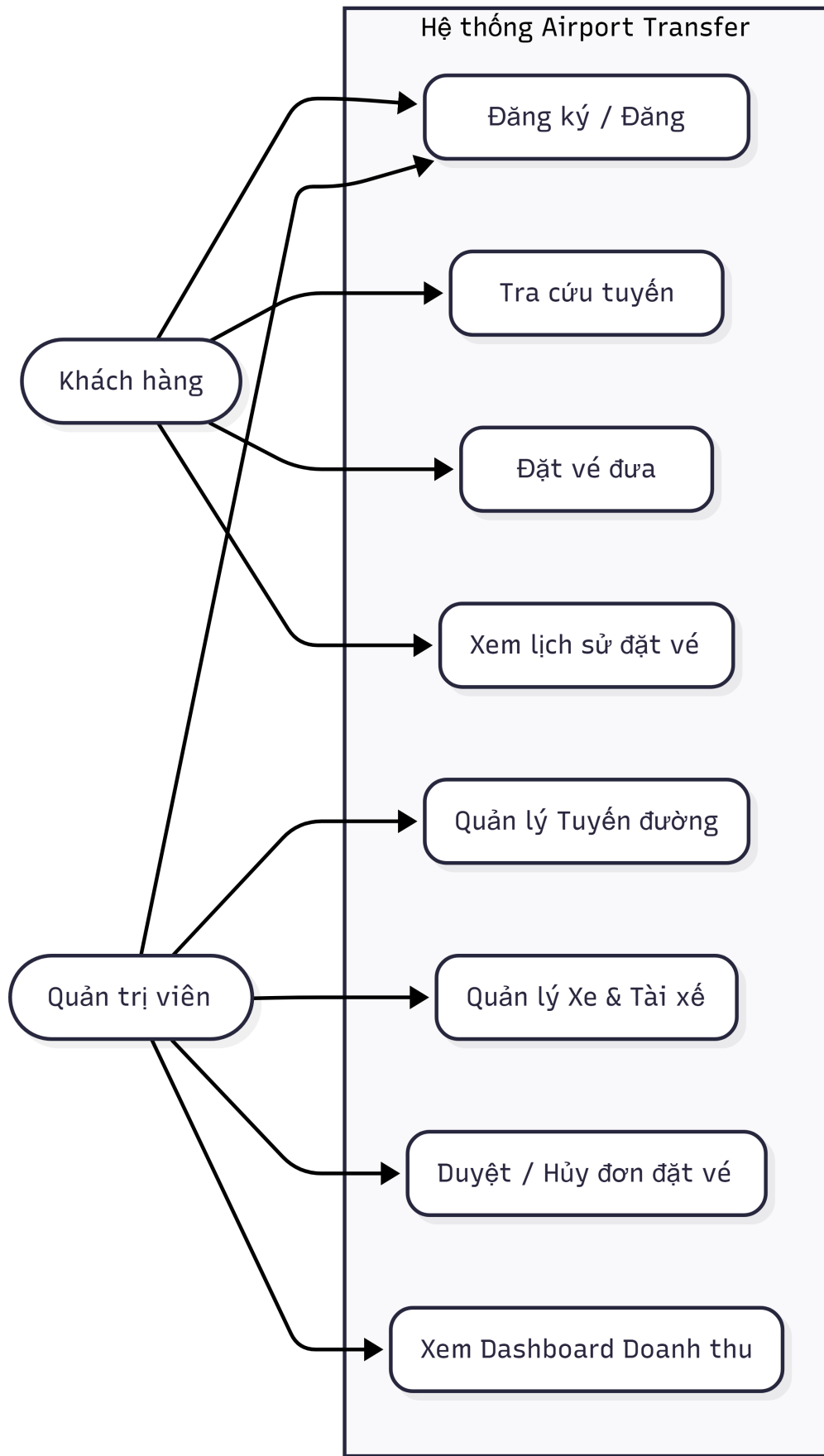
Dựa trên cấu trúc phân phối đường dẫn hệ thống (Routing Framework Architecture), mô hình nghiệp vụ được phân hóa thành ba nhóm tác nhân chính tương tác trực tiếp với các tài nguyên phần mềm thông qua các giao thức mạng bảo mật: Khách vắng lai (Guest), Khách hàng đã xác thực (Authenticated User) và Quản trị viên hệ thống (System Administrator).

2.1.1 Xác định và phân loại Tác nhân (Actors)

- **Khách vắng lai (Guest):** Là những người dùng chưa thực hiện định danh hoặc phiên làm việc (Session) chưa được thiết lập trên hệ thống. Nhóm tác nhân này chỉ có quyền tiếp cận các tài nguyên tĩnh hoặc động mang tính chất công khai, không nhạy cảm như tra cứu tuyến đường, xem chi tiết lộ trình vận chuyển, thực hiện đăng ký tài khoản mới hoặc đăng nhập vào hệ thống.
- **Khách hàng đã xác thực (Authenticated User):** Là tác nhân người dùng phổ thông đã vượt qua màn lọc xác thực (auth middleware) và được cấp phát một Session ID hợp lệ. Ngoài việc kế thừa toàn bộ quyền hạn của Khách vắng lai, nhóm người dùng này được phép tương tác với cơ sở dữ liệu để khởi tạo đơn đặt xe (Booking Requests), theo dõi lịch trình cá nhân và kiểm tra lịch sử phân trang trạng thái đơn hàng của chính mình.
- **Quản trị viên (Admin):** Là tác nhân tối cao sở hữu thuộc tính quyền hạn đặc biệt (`role = 'admin'`) trong bảng dữ liệu thực thể. Tác nhân này được bảo vệ nghiêm ngặt bởi màn lọc kiểm soát truy cập chuyên biệt (AdminMiddleware), có toàn quyền can thiệp vào vòng đời của dữ liệu hệ thống (CRUD) bao gồm phương tiện, tài xế, tuyến xe và nắm giữ quyền sinh sát trong việc kiểm duyệt, phê duyệt hoặc hủy bỏ đơn đặt chuyến của khách hàng.

2.1.2 Thiết kế sơ đồ Use Case tổng thể

Dưới đây là cấu trúc thiết kế biểu đồ các ca sử dụng phân rã chức năng để lập trình viên hiện thực hóa trên môi trường Overleaf:



Hình 2.1: Biểu đồ Use Case tổng thể phân rã các phân hệ chức năng

2.1.3 Đặc tả chi tiết các Use Case trọng tâm

Để hiện thực hóa mã nguồn một cách chính xác tại tầng xử lý logic, các ca sử dụng cốt lõi của hệ thống cần được đặc tả cấu trúc luồng sự kiện một cách tường minh:

1. Ca sử dụng: Tra cứu và Lọc tuyến xe (Search/Filter Routes)

- *Tác nhân kích hoạt:* Guest, Authenticated User.
- *Tiền điều kiện:* Hệ thống phải có các tuyến xe được cấu hình trạng thái 'active'.
- *Luồng sự kiện chính:* Người dùng nhập điểm đón (pickup_point) hoặc điểm trả (dropoff_point) tại giao diện. Hệ thống tiếp nhận HTTP Request gửi kèm tham số truy vấn, sử dụng cơ chế so khớp chuỗi tương đối (LIKE %keyword%) thông qua câu lệnh điều kiện của trình ORM để truy xuất dữ liệu từ bảng transfer_routes, thực hiện phân trang cố định và trả về kết quả hiển thị cho người dùng.

2. Ca sử dụng: Đặt chuyến xe đưa đón (Create Booking)

- *Tác nhân kích hoạt:* Authenticated User.
- *Tiền điều kiện:* Người dùng đã đăng nhập thành công và tuyến xe lựa chọn phải có trạng thái hoạt động bình thường.
- *Luồng sự kiện chính:* Người dùng điền biểu mẫu (Form) bao gồm họ tên hành khách, số điện thoại, thời gian đón (pickup_time), số lượng người đi. Hệ thống tiến hành xác thực dữ liệu đầu vào thông qua lớp kiểm thử (StoreBookingRequest). Nếu dữ liệu hợp lệ, tầng Logic sẽ nhân số lượng khách với giá vé gốc của tuyến để áp đặt giá trị tổng tiền (total_price), ghi dữ liệu xuống bảng bookings với trạng thái mặc định ban đầu là 'pending' và chuyển hướng người dùng về trang lịch sử cá nhân.

3. Ca sử dụng: Điều phối và Phê duyệt đơn đặt xe (Approve/Cancel Booking)

- *Tác nhân kích hoạt:* Admin.
- *Tiền điều kiện:* Tài khoản điều hành phải vượt qua bộ lọc quyền hạn cấp cao; đơn hàng cần xử lý phải ở trạng thái 'pending'.
- *Luồng sự kiện chính:* Admin truy cập màn hình quản trị đơn đặt vé, lựa chọn một đơn cụ thể. Khi bấm chọn phê duyệt (confirm) hoặc hủy bỏ (cancel), hệ thống sẽ gửi một PATCH Request lên máy chủ. Bộ điều khiển kiểm tra điều kiện logic trạng thái hiện tại của đơn hàng. Nếu đơn hàng vẫn ở trạng thái chờ xử lý, hệ thống cập nhật trường status thành 'confirmed' hoặc 'cancelled' tương ứng, tạo thông báo flash báo tin thành công và kết xuất lại giao diện bảng.

2.2 Phân tích và Thiết kế Phân hệ Chức năng

Hệ thống được module hóa thành ba phân hệ chức năng độc lập về mặt logic nhưng có sự liên kết chặt chẽ về mặt chia sẻ tài nguyên dữ liệu thông qua kiến trúc máy chủ tập trung.

2.2.1 Phân hệ Xác thực và Định danh (Authentication Module)

Phân hệ này chịu trách nhiệm thiết lập hàng rào an ninh sinh học đầu tiên cho ứng dụng Web, chịu trách nhiệm quản lý toàn bộ vòng đời phiên làm việc của tài khoản dựa trên cấu trúc lõi bảo mật mã nguồn. Các chức năng thành phần bao gồm:

- **Đăng ký tài khoản (User Registration):** Tiếp nhận các thuộc tính name, email, password từ Client. Hệ thống áp dụng thuật toán băm bảo mật một chiều BCrypt với số vòng mã hóa tiêu chuẩn (rounds = 12) để chuyển đổi mật khẩu thuần sang dạng chuỗi ký tự mã hóa an toàn trước khi lưu trữ vào đĩa cứng, triệt tiêu hoàn toàn nguy cơ lộ mật khẩu ngay cả khi cơ sở dữ liệu bị chiếm đoạt.

- **Đăng nhập và Tạo phiên (User Login & Session Management):** Đối sánh thông tin tài khoản thu thập từ biểu mẫu với dữ liệu lưu trữ. Nếu trùng khớp, cơ chế `session()` -> `regenerate()` của máy chủ lập tức được kích hoạt nhằm làm mới Session ID của phiên làm việc, ngăn chặn triệt để lỗ hổng tấn công cố định phiên (Session Fixation Attack) nguy hiểm.
- **Xác minh và Khôi phục (Email Verification & Password Reset):** Tạo lập các Token ngẫu nhiên kết hợp với chữ ký điện tử (Signed URLs) có giới hạn thời gian tồn tại và cơ chế giới hạn tần suất yêu cầu (`throttle: 6, 1` middleware - tối đa 6 yêu cầu trong 1 phút) để xử lý luồng lấy lại mật khẩu và gửi email xác minh thông tin tài khoản trực tuyến.

2.2.2 Phân hệ Khách hàng (User Web Interface)

Phân hệ này tập trung cung cấp không gian trải nghiệm tối giản, trực quan hóa luồng dữ liệu nghiệp vụ cho khách hàng có nhu cầu di chuyển. Trục tính năng bao gồm:

- **Trang chủ và Tra cứu tuyến xe:** Hiển thị danh sách lưới phân trang (9 tuyến đường trên mỗi trang) chứa hình ảnh trực quan, điểm xuất phát, điểm kết thúc và giá cả niêm yết công khai. Tích hợp bộ công cụ lọc tìm kiếm động phản hồi nhanh ngay tại Client.
- **Trang Đặt vé chuyển xe:** Giao diện form nhập liệu thông tin hành khách an toàn, tự động liên kết ID của người dùng đăng nhập hiện tại với thực thể chuyến đi được chọn, ngăn ngừa các hành vi tiêm nhiễm dữ liệu sai lệch từ bên ngoài.
- **Trang Quản lý lịch sử cá nhân:** Bảng dữ liệu phân trang hiển thị toàn bộ lịch trình người dùng đã thực hiện đặt chỗ trong quá khứ và hiện tại. Trạng thái đơn hàng được trực quan hóa bằng các nhãn màu (Badge Component) sắc nét để người dùng tiện theo dõi tiến độ phê duyệt của nhà xe.

2.2.3 Phân hệ Quản trị viên (Admin Command Control)

Là trung tâm đầu não điều khiển luồng hoạt động kinh doanh tổng thể của toàn ứng dụng, bao gồm các trục xử lý nghiệp vụ nặng:

- **Bảng điều khiển Thống kê tổng hợp (Dashboard Analysis):** Khi Admin đăng nhập thành công vào trang tổng quan, một chuỗi các câu lệnh truy vấn hàm tích hợp (Count, Sum) của cơ sở dữ liệu sẽ tự động thực thi tại tầng Server để tổng hợp các thông số kinh tế cốt lõi: Tổng số lượt đặt vé, số lượng đơn đang nghẽn chờ duyệt, tổng doanh thu tài chính thực tế thu về từ các đơn đã hoàn tất. Đồng thời sử dụng thư viện xử lý thời gian Carbon để thu thập dữ liệu doanh thu của 7 ngày liên tiếp gần nhất phục vụ vẽ biểu đồ trực quan.
- **Quản lý danh mục Tài nguyên (CRUD Resources Management):** Cung cấp đầy đủ các giao diện biểu mẫu Thêm/Sửa/Xóa đối với ba thực thể nền tảng: Phương tiện (Vehicles), Tài xế (Drivers), và Tuyến xe chạy (TransferRoutes). Phân hệ tích hợp luồng xử lý tệp tin đa phương tiện (File Upload Processing), lưu trữ ảnh xe hoặc ảnh tuyến vào thư mục lưu trữ cục bộ bảo mật công khai (`storage/app/public/images`) và chỉ lưu đường dẫn chuỗi (String path) xuống database để tối ưu hóa dung lượng lưu trữ của hệ quản trị CSDL.
- **Trung tâm duyệt đơn tập trung:** Giao diện quản lý danh sách đơn hàng toàn hệ thống, hỗ trợ lọc đơn hàng theo trạng thái và cung cấp các nút kích hoạt luồng đổi trạng thái tức thời kèm cơ chế cảnh báo Javascript tại trình duyệt để tránh thao tác nhầm lẫn.

2.3 Thiết kế Cơ sở dữ liệu lô-gíc (Database Schema)

Để đảm bảo hệ thống phần mềm vận hành với hiệu năng cao, tránh hiện tượng thắt nút cổ chai (Bottleneck) khi đọc ghi dữ liệu và duy trì tính toàn vẹn tham chiếu, cơ sở dữ liệu quan hệ (RDBMS) của hệ thống Airport Transfer được

thiết kế chuẩn hóa đạt cấp độ 3NF. Toàn bộ lược đồ thực thể và mối quan hệ ràng buộc vật lý được khai báo thông qua cấu trúc mã nguồn di trú dữ liệu (Database Migrations).

2.3.1 Định nghĩa cấu trúc chi tiết các bảng dữ liệu

Dưới đây là đặc tả chi tiết kiểu dữ liệu, độ dài và các ràng buộc toàn vẹn của các bảng cốt lõi trong hệ thống:

1. Bảng users (Quản lý tài khoản người dùng và Admin)

Lưu trữ thông tin định danh và phân quyền đặc quyền của hệ thống.

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả nghĩa nghiệp vụ
id	Unsigned BigInt	Primary Key, Auto Inc	Khóa chính định danh tự tăng
name	Varchar(255)	Not Null	Họ và tên chủ tài khoản
email	Varchar(255)	Unique, Not Null	Địa chỉ Email (Dùng đăng nhập)
email_verified_at	Timestamp	Nullable	Thời gian xác minh tài khoản
password	Varchar(255)	Not Null	Chuỗi mật khẩu đã băm (BCrypt)
role	Enum('user', 'admin')	Default 'user'	Vai trò phân quyền trong hệ thống
remember_token	Varchar(100)	Nullable	Chuỗi Token duy trì trạng thái đăng nhập
timestamps	Timestamp	Not Null	Tự động tạo trường created_at, updated_at

Bảng 2.1: Cấu trúc chi tiết bảng dữ liệu users

2. Bảng vehicles (Quản lý tài nguyên phương tiện vận tải)

Lưu trữ thông tin kỹ thuật của đội xe đưa đón sân bay. Hỗ trợ cơ chế xóa mềm để bảo toàn dữ liệu lịch sử.

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả nghĩa nghiệp vụ
id	Unsigned BigInt	Primary Key, Auto Inc	Khóa chính phương tiện
name	Varchar(255)	Not Null	Tên dòng xe/Tên phương tiện
license_plate	Varchar(20)	Unique, Not Null	Biển số kiểm soát xe
seats	Unsigned TinyInt	Not Null	Số lượng ghế ngồi thiết kế cố định
image	Varchar(255)	Nullable	Đường dẫn file hình ảnh đại diện xe
status	Enum(...)	Active, Inactive, Maintenance	Trạng thái vận hành hiện tại
deleted_at	Timestamp	Nullable	Thời gian xóa mềm (SoftDeletes)
timestamps	Timestamp	Not Null	Thời gian tạo lập và cập nhật bản ghi

Bảng 2.2: Cấu trúc chi tiết bảng dữ liệu vehicles

3. Bảng drivers (Quản lý hồ sơ nhân sự tài xế)

Quản lý thông tin liên hệ và giấy phép hoạt động của đội ngũ tài xế đường dài.

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả nghĩa nghiệp vụ
id	Unsigned BigInt	Primary Key, Auto Inc	Khóa chính định danh tài xế
name	Varchar(255)	Not Null	Họ và tên đầy đủ của tài xế
phone	Varchar(20)	Not Null	Số điện thoại di động liên lạc
license_number	Varchar(50)	Not Null	Số Giấy phép lái xe (GPLX) quân sự/dân sự
status	Enum('active', 'inactive')	Default 'active'	Trạng thái sẵn sàng nhận chuyển
deleted_at	Timestamp	Nullable	Thời gian xóa mềm phục vụ lưu trữ
timestamps	Timestamp	Not Null	Ghi nhận thời gian tạo và sửa đổi

Bảng 2.3: Cấu trúc chi tiết bảng dữ liệu drivers

4. Bảng transfer_routes (Quản lý danh mục Tuyến đường và Lộ trình xe chạy)

Bảng này đóng vai trò là thực thể trung tâm kết nối, cấu hình cố định xe và tài xế phụ trách cho từng cung đường sân bay cụ thể.

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả nghĩa nghiệp vụ
id	Unsigned BigInt	Primary Key, Auto Inc	Khóa chính định danh lộ trình
name	Varchar(255)	Not Null	Tên gọi tuyến đường hiển thị trực quan
pickup_point	Varchar(255)	Not Null	Địa điểm đón khách hàng cụ thể
dropoff_point	Varchar(255)	Not Null	Địa điểm trả khách hàng tại đích
price	Decimal(10,2)	Not Null	Giá vé cơ sở áp dụng cho một hành khách
duration_minutes	Unsigned Int	Not Null	Thời gian di chuyển dự kiến (Tính bằng phút)
description	Text	Nullable	Mô tả các tiện ích đi kèm của tuyến
image	Varchar(255)	Nullable	Ảnh nền đại diện cho tuyến xe
status	Enum('active', 'inactive')	Default 'active'	Trạng thái mở/đóng tuyến của nhà xe
vehicle_id	Unsigned BigInt	Foreign Key, Nullable	Liên kết khóa ngoại bảng vehicles
driver_id	Unsigned BigInt	Foreign Key, Nullable	Liên kết khóa ngoại bảng drivers
deleted_at	Timestamp	Nullable	Khai báo xóa mềm toàn vẹn quan hệ
timestamps	Timestamp	Not Null	Thời gian khởi tạo dữ liệu

Bảng 2.4: Cấu trúc chi tiết bảng dữ liệu transfer_routes

5. Bảng bookings (Quản lý các giao dịch đặt chỗ và vé chuyến)

Lưu trữ toàn bộ thông tin đơn hàng do người dùng khởi tạo, thực hiện tham chiếu chéo phụ thuộc dữ liệu chéo giữa khách hàng và lộ trình xe tương ứng.

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả nghĩa nghiệp vụ
id	Unsigned BigInt	Primary Key, Auto Inc	Khóa chính hóa đơn đặt vé
user_id	Unsigned BigInt	Foreign Key, Constrained	Mã khách hàng (Liên kết bảng users)
transfer_route_id	Unsigned BigInt	Foreign Key, Constrained	Mã tuyến xe (Liên kết bảng transfer_routes)
passenger_name	Varchar(255)	Not Null	Họ tên hành khách di chuyển thực tế
passenger_phone	Varchar(20)	Not Null	Số điện thoại hành khách nhận SMS điều xe
pickup_time	DateTime	Not Null	Thời gian chính xác xe đến điểm hẹn đón
num_passengers	Unsigned TinyInt	Not Null	Số lượng người đi cùng trong nhóm
total_price	Decimal(10,2)	Not Null	Tổng chi phí hóa đơn (Đã nhân số người)
note	Text	Nullable	Ghi chú yêu cầu đặc biệt của khách hàng
status	Enum(...)	Pending, Confirmed, Cancelled	Trạng thái vòng đời của đơn đặt vé
timestamps	Timestamp	Not Null	Thời gian phát sinh giao dịch đặt chỗ

Bảng 2.5: Cấu trúc chi tiết bảng dữ liệu bookings

2.3.2 Mô hình Quan hệ Thực thể liên kết (Entity-Relationship Diagram)

Dữ liệu của hệ thống Airport Transfer là một chuỗi liên kết các ràng buộc chặt chẽ. Lược đồ ERD phản ánh cấu trúc logic sau:

- **Mối quan hệ users – bookings (Quan hệ 1 – Nhiều):** Một tài khoản người dùng có thể thực hiện đặt nhiều đơn đặt vé xe sân bay khác nhau qua thời gian (hasMany), nhưng một đơn hàng cụ thể bắt buộc chỉ thuộc quyền sở hữu độc quyền của một tài khoản người dùng duy nhất (belongsTo).
- **Mối quan hệ transfer_routes – bookings (Quan hệ 1 – Nhiều):** Một lộ trình xe chạy đưa đón sân bay có thể tiếp nhận nhiều đơn đặt chỗ từ các nhóm hành khách khác nhau trong các khung giờ khác nhau, một đơn đặt xe bắt buộc phải ánh xạ cụ thể vào một lộ trình cố định.
- **Mối quan hệ thành phần tài nguyên (Quan hệ vehicles/drivers – transfer_routes):** Một tài xế hoặc một phương tiện tại một thời điểm vận hành được cấu hình cố định cho một tuyến đường cụ thể của Admin nhằm duy trì tính nhất quán, giảm thiểu việc tính toán xếp lịch động quá tải tài nguyên máy chủ.

2.4 Sơ đồ Luồng Xử lý Nghiệp vụ Hệ thống (Request Lifecycle)

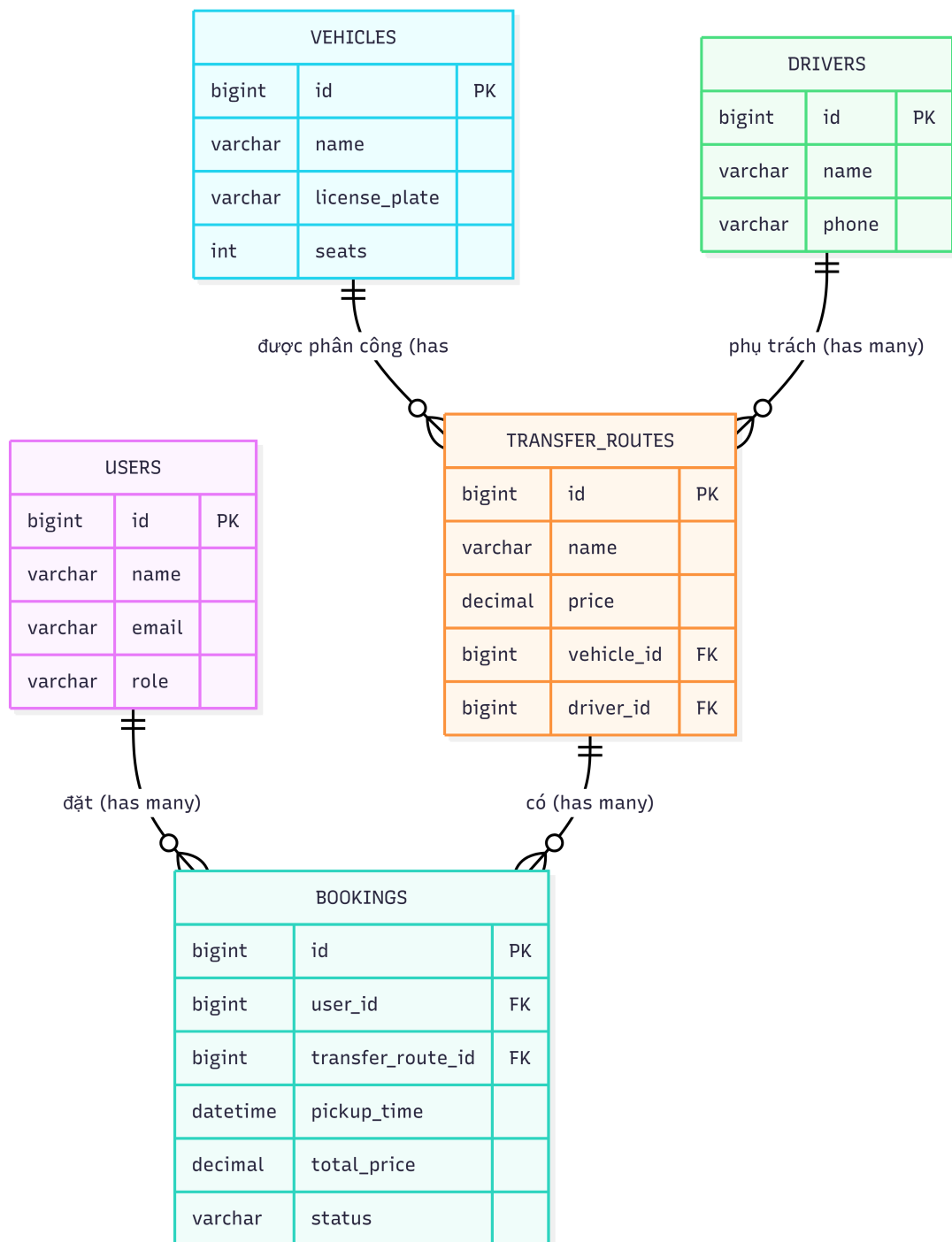
Để hiểu rõ cách thức mã nguồn ứng dụng Laravel xử lý dữ liệu và điều hướng luồng thông tin khi nhận được yêu cầu từ trình duyệt Client, chúng ta tiến hành phân tích Sơ đồ luồng xử lý vòng đời của một yêu cầu HTTP (HTTP Request Lifecycle). Luồng tương tác nghiệp vụ được mô tả thông qua quy trình xử lý tuần tự qua các tầng kiến trúc:

1. **Giai đoạn Đánh chặn và Định tuyến (Routing & Middleware Phase):** Khi người dùng gửi một hành động từ Client (Ví dụ: Yêu cầu phê duyệt đơn hàng bằng cách nhấn nút "Duyệt" tại màn hình quản trị), một HTTP Request mang phương thức PATCH hướng tới địa chỉ URL /admin/bookings/{id}/confirm được gửi đi. Tập tin định tuyến hệ thống routes/web.php tiếp nhận yêu cầu, phân tích tiền tố đường dẫn và lập tức chuyển dữ liệu qua màng lọc bảo mật AdminMiddleware. Tại đây, hệ thống đối sánh kiểm tra nếu người dùng chưa đăng nhập hoặc thuộc tính role không phải là 'admin', luồng xử lý bị bẻ gãy ngay lập tức qua lệnh abort(403). Ngược lại, yêu cầu hợp lệ sẽ được chuyển tiếp vào bộ điều khiển Admin\BookingController.
2. **Giai đoạn Xử lý Logic và Tương tác Mô hình (Controller & Model Phase):** Bộ điều khiển tiếp nhận tham số ID thực thể thông qua cơ chế ngầm định ánh xạ Eloquent Model (Route Model Binding). Hàm confirm(Booking \$booking) trích xuất bản ghi trực tiếp từ MySQL. Trước khi ghi dữ liệu, controller thực hiện kiểm tra điều kiện ràng buộc logic nghiệp vụ: if (\$booking->status !== 'pending') thì lập tức quay đầu và đẩy thông báo cảnh báo lỗi flash. Nếu điều kiện thỏa mãn, controller gọi hàm \$booking->update(['status' => 'confirmed']) đẩy lệnh ghi xuống database cập nhật trạng thái đơn hàng bền vững.
3. **Giai đoạn Kết xuất phản hồi (View Rendering & HTTP Response):** Sau khi dữ liệu được cập nhật thành công dưới CSDL MySQL, bộ điều khiển thực hiện chuyển hướng phản hồi redirect()->back() kèm theo chuỗi thông điệp flash báo tin thành công lưu trữ ngắn hạn trong Session. Tầng hiển thị giao diện Blade View nhận được dữ liệu phản hồi, thành phần linh kiện giao diện <x-ui.alert> tự động biên dịch mã HTML kết hợp với các class Tailwind CSS hiển thị thông báo chuyển màu xanh lá trực quan sinh động trên màn hình người dùng, khép lại hoàn chỉnh vòng đời xử lý yêu cầu của hệ thống.

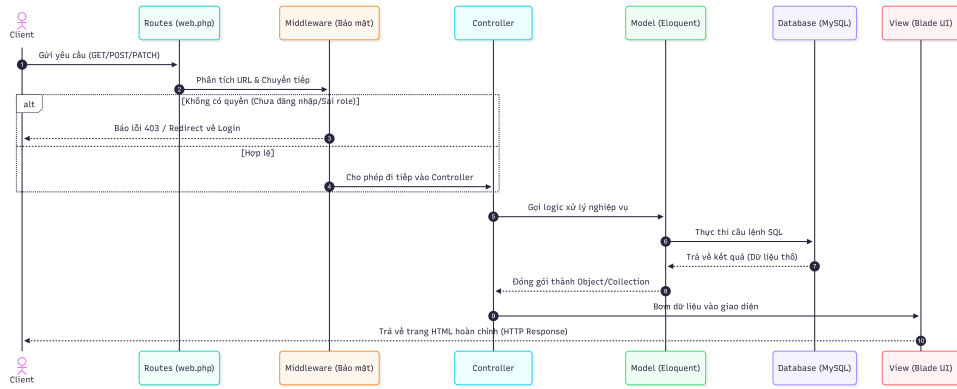
Chương 3: THIẾT KẾ CHI TIẾT VÀ HIỆN THỰC HÓA MÃ NGUỒN

3.1 Hiện thực hóa kiến trúc Laravel MVC trong dự án

Để chuyển đổi các yêu cầu phân tích hệ thống thành một sản phẩm phần mềm chạy được trên môi trường thực tế, dự án đã hiện thực hóa mô hình kiến trúc Model-View-Controller (MVC) thông qua các lớp xử lý và tập tin mã nguồn



Hình 2.2: Sơ đồ mối quan hệ giữa các thực thể vật lý dưới cơ sở dữ liệu (ERD)



Hình 2.3: Sơ đồ luồng tuần tự xử lý yêu cầu nghiệp vụ phía Server

cụ thể trong Framework Laravel. Việc thiết lập này tuân thủ nguyên lý tách biệt mã nguồn xử lý logic và mã kết xuất đồ họa giao diện, giúp đơn giản hóa quá trình bảo trì và kiểm thử.

3.1.1 Thành phần Model và Cơ chế ánh xạ quan hệ dữ liệu

Tầng mã nguồn Mô hình dữ liệu (Model Layer) nằm tại thư mục `app/Models/`, đóng vai trò đại diện trực tiếp và định nghĩa toàn bộ quy tắc ràng buộc, thuộc tính của các bảng vật lý trong MySQL. Hệ thống đã khai báo năm lớp đối tượng cốt lõi tương ứng với năm thực thể kinh doanh của dịch vụ đặt xe sân bay:

- `User.php`: Kế thừa từ lớp `Authenticatable` của Laravel để cung cấp các thuộc tính bảo mật tài khoản. Thuộc tính `$fillable` định nghĩa các trường cho phép ghi dữ liệu hàng loạt (mass assignment) gồm `name`, `email`, `password`, và `role`. Lớp này chứa hàm `bookings()` thiết lập mối quan hệ một-nhiều với thực thể đơn đặt chỗ thông qua mã lệnh `return $this->hasMany(Booking::class);`.
- `Booking.php`: Định nghĩa thực thể đơn giao dịch đặt chỗ. Lớp này bọc thuộc tính cấu hình `$casts` để ép kiểu dữ liệu chuỗi thời gian `pickup_time` tự động chuyển sang đối tượng `DateTime` (Carbon instance) trong PHP, giúp lập trình viên thao tác định dạng ngày tháng mượt mà ở tầng hiển thị. Thực thể này liên kết ngược lại (Inverse Relationships) thông qua hai hàm khóa ngoại: `user()` (`belongsTo` với lớp `User`) và `transferRoute()` (`belongsTo` với lớp `TransferRoute`).
- `TransferRoute.php`: Đại diện cho thông tin tuyến lộ trình vận chuyển. Lớp này định nghĩa mối quan hệ một-nhiều phụ thuộc với bảng `Booking`, đồng thời sở hữu hai mối quan hệ liên kết ngược `belongsTo` hướng tới model phương tiện xe (`vehicle`) và nhân sự tài xế (`driver`).
- `Vehicle.php` và `Driver.php`: Đại diện cho danh mục phương tiện xe và hồ sơ tài xế. Cả hai lớp này đều tích hợp thuộc tính đặc trưng `use SoftDeletes`; kết hợp với thư viện `Illuminate\Database\Eloquent\SoftDeletes`. Đây là kỹ thuật kỹ nghệ phần mềm quan trọng, đảm bảo rằng khi Admin thực hiện xóa một chiếc xe hoặc một tài xế ra khỏi hệ thống, dữ liệu không bị xóa vĩnh viễn khỏi ổ cứng mà chỉ cập nhật mốc thời gian vào trường `deleted_at`. Nhờ vậy, toàn bộ hồ sơ lịch sử đơn hàng cũ thuộc bảng `bookings` tham chiếu tới ID của xe và tài xế đó vẫn tồn tại vẹn toàn, không gây ra lỗi sụp đổ liên kết dữ liệu mồ côi (Orphan Records).

3.1.2 Thành phần Controller và Phân tách không gian xử lý nghiệp vụ

Tầng điều khiển trung tâm (Controller Layer) nằm tại thư mục `app/Http/Controllers/`, chịu trách nhiệm xử lý toàn bộ luồng nghiệp vụ của ứng dụng phần mềm. Để tránh việc viết mã nguồn chồng chéo phụ thuộc lẫn nhau, dự án tiến hành phân chia cấu trúc không gian tên (Namespaces) và thư mục lưu trữ thành các phân hệ riêng biệt:

1. **Phân hệ xác thực nằm tại thư mục** `app/Http/Controllers/Auth/`: Quản lý các bộ điều khiển logic nhỏ lẻ theo triết lý Single Responsibility của Laravel Breeze như `AuthenticatedSessionController.php` (điều khiển luồng đăng nhập), `RegisteredUserController.php` (điều khiển luồng tạo tài khoản mới), hay `PasswordController.php` (xử lý đổi mật khẩu).
2. **Phân hệ khách hàng nằm tại thư mục** `app/Http/Controllers/User/`: Gồm `RouteController.php` dùng để hiển thị và tìm kiếm lộ trình cho hành khách và `BookingController.php` xử lý luồng đặt chuyến xe. Tại đây, hàm `store()` của bộ điều khiển đặt vé thể hiện rõ tư duy bảo mật nghiệp vụ vững chắc: Hệ thống ép buộc dữ liệu đi qua màng lọc kiểm thử `StoreBookingRequest`, sau đó tự động truy vấn giá vé gốc của chuyến xe trực tiếp từ cơ sở dữ liệu phía Server thông qua câu lệnh `TransferRoute::findOrFail(...)` rồi mới tiến hành nhân với số lượng hành khách để tính ra `total_price`. Logic này chặn đứng hoàn toàn nguy cơ người dùng cố tình can thiệp vào mã nguồn Front-end để thay đổi biến số giá tiền (Price Tampering) trước khi gửi Request lên máy chủ.
3. **Phân hệ điều hành nằm tại thư mục** `app/Http/Controllers/Admin/`: Triển khai cấu trúc bộ điều khiển tài nguyên chuẩn RESTful (Resource Controllers) để xử lý các bảng dữ liệu tĩnh. Điển hình là `DashboardController.php`, chịu trách nhiệm tổng hợp các chỉ số kinh tế lớn bằng các hàm kết hợp tối ưu để đẩy lên màn hình trung tâm của ban điều hành.

3.1.3 Thành phần View và Cơ chế xây dựng linh kiện giao diện nâng cao

Tầng hiển thị giao diện người dùng (View Layer) nằm tại thư mục `resources/views/`, được hiện thực hóa dựa trên Blade Template Engine của Laravel kết hợp phong cách Utility-first của Tailwind CSS. Thay vì viết mã nguồn giao diện dưới dạng các trang HTML độc lập kéo dài hàng ngàn dòng gây lộn xộn code bản, dự án đã áp dụng phương pháp thiết kế hướng linh kiện (Component-Driven Development):

- **Cơ chế kế thừa khung giao diện chính (Layout Inheritance)**: Dự án định nghĩa tệp tin `admin-layout.blade.php` và `user.blade.php` đóng vai trò là vỏ bọc khung sườn (Shell Layouts). Các tệp tin này chứa các thẻ liên kết CSS/JS tĩnh biên dịch bởi Vite, cấu hình phông chữ nền 'Be Vietnam Pro' và chèn thẻ vị trí động `{slot}`. Các tệp hiển thị con chỉ cần gọi thẻ bọc `<x-admin-layout>` hoặc `<x-user-layout>`, toàn bộ nội dung bên trong sẽ tự động được tiêm vào vị trí quy định, giúp tái sử dụng cấu trúc giao diện lên tới 90%.
- **Xây dựng hệ thống linh kiện UI nguyên tử (Atomic View Components)**: Nằm trong thư mục thành phần, dự án tạo lập các file giao diện siêu nhỏ để đại diện cho các thành phần UI dùng chung:
 - `input.blade.php`: Linh kiện bọc nhãn `<label>` và thẻ nhập liệu `<input>`, tích hợp sẵn chỉ thị `@error('name')` của Laravel để tự động đổ viền màu đỏ kèm chuỗi văn bản cảnh báo lỗi khi biểu mẫu không vượt qua màng lọc xác thực.
 - `badge.blade.php`: Linh kiện nhãn trạng thái trực quan, sử dụng mảng ánh xạ PHP tương ứng: 'green' ánh xạ chuỗi class `'bg-green-100 text-green-800'`, 'red' ánh xạ `'bg-red-100 text-red-800'`. Khi kết xuất giao diện đơn hàng, mã nguồn chỉ cần gọi `<x-ui.badge:color="...">`, hệ thống sẽ tự động hiển thị các khối màu sắc nét tương ứng với trạng thái đơn hàng (Pending - Vàng, Confirmed - Xanh lá, Cancelled - Đỏ) mà không cần viết lại các điều kiện rẽ nhánh CSS phức tạp.
 - `modal.blade.php`: Linh kiện hộp thoại tương tác nổi, tận dụng thuộc tính khai báo `x-data` của Alpine.js để điều khiển trạng thái ẩn hiện trực tiếp tại Client, triệt tiêu việc phải viết các đoạn mã JQuery dài dòng.

3.2 Hiện thực hóa các Module chức năng cốt lõi

3.2.1 Module Xác thực hệ thống và Màng lọc phân quyền nâng cao

Luồng xác thực tài khoản của dự án được cấu hình định tuyến thông qua tệp tin `routes/auth.php`. Tiến trình kiểm soát an ninh phân quyền được hiện thực hóa thông qua tệp mã nguồn bộ lọc trung gian `app/Http/Middleware/AdminMiddleware.php`. Bản chất kiến trúc hoạt động của lớp màng lọc này được thể hiện qua đoạn mã xử lý lỗi sau:

```
public function handle(Request $request, Closure $next): Response
{
    if (! $request->user() ||
        $request->user()->role !== 'admin') {
        abort(403);
    }
    return $next($request);
}
```

Khi một luồng yêu cầu HTTP Request hướng dịch vụ tới nhóm đường dẫn quản trị được bọc bởi tiền tố `Route::middleware(['auth', 'admin'])->prefix('admin')`, máy chủ sẽ kích hoạt hàm `handle()` của `AdminMiddleware`. Phương thức `$request->user()` truy xuất trực tiếp thực thể tài khoản đang liên kết với Session ID hiện tại. Nếu giá trị của thuộc tính `role` khác chuỗi ký tự `'admin'`, hệ thống sẽ ngay lập tức thực hiện ngắt luồng xử lý bằng lệnh `abort(403)`, trả về mã trạng thái lỗi từ chối truy cập tiêu chuẩn của giao thức HTTP, bảo vệ tuyệt đối các hàm CRUD tài nguyên phía sau khỏi các hành vi dò tìm hoặc tấn công leo thang đặc quyền đường dẫn thẳng URL.

3.2.2 Module Quản lý Tuyến xe và Đặt vé chuyển phía Khách hàng

Nghiệp vụ cốt lõi dành cho đối tượng hành khách được đảm nhiệm bởi sự phối hợp nhịp nhàng giữa `User\RouteController` và `User\BookingController`.

- **Cơ chế tìm kiếm và phân trang tối ưu lộ trình:** Tại trang danh sách tuyến đường, để xử lý bài toán tìm kiếm xe vượt mà khi lượng dữ liệu lớn, hàm `index()` tiếp nhận hai tham số lọc là điểm đón `pickup_point` và điểm trả `dropoff_point`. Bộ điều khiển sử dụng cấu trúc xây dựng câu lệnh điều kiện lồng của Eloquent (`when()` closure):

```
$routes = TransferRoute::where('status', 'active')
->when($searchPickup, fn ($q) =>
    $q->where(
        'pickup_point',
        'like',
        "%{$searchPickup}%"
    )
)
->when($searchDropoff, fn ($q) =>
    $q->where(
        'dropoff_point',
        'like',
        "%{$searchDropoff}%"
    )
)
```

```
->latest()
->paginate(9);
```

Cơ chế này đảm bảo chỉ những tuyến xe có trạng thái 'active' mới được hiển thị. Câu lệnh `paginate(9)` tự động biên dịch câu truy vấn SQL kèm mệnh đề `LIMIT` và `OFFSET` tương ứng để trích xuất đúng 9 bản ghi trên một trang, giảm thiểu gánh nặng bộ nhớ RAM của máy chủ database. Đồng thời, tệp view `index.blade.php` phía Client sử dụng hàm `$routes->links()` để tự động kết xuất các nút chuyển trang thông minh (Trước/Sau), duy trì tham số tìm kiếm cũ qua hàm `withQueryString()`.

- **Quy trình khởi tạo đơn hàng an toàn tài chính:** Khi người dùng bấm chọn một tuyến xe hợp lệ và gửi biểu mẫu đăng ký thông tin hành khách lên hàm `store()`, hệ thống bọc luồng ghi dữ liệu trong lớp kiểm thử dữ liệu đầu vào nghiêm ngặt để tránh lỗi SQL. Sau khi tính toán tổng chi phí dựa trên giá vé hệ thống nhân số khách, hàm `Booking::create([...])` được kích hoạt, tự động gán giá trị chuỗi mặc định 'status' => 'pending' và trả về thông báo flash thành công để điều hướng hành khách quay lại trang quản lý lịch sử cá nhân công khai.

3.2.3 Module Quản lý tài nguyên Đội xe và Phê duyệt đơn hàng phía Admin

Phân hệ quản trị cao cấp của ban điều hành tập trung giải quyết bài toán quản lý tài nguyên tính và kiểm duyệt dòng tiền hóa đơn đặt vé:

- **Kỹ thuật xử lý tệp tin đa phương tiện và lưu trữ bất đối xứng:** Trong các bộ điều khiển Admin \RouteController và Admin\VehicleController, các phương thức khởi tạo và cập nhật (`store()` / `update()`) đều tích hợp luồng xử lý ảnh tải lên từ biểu mẫu của Admin. Mã nguồn kiểm tra điều kiện `if ($request->hasFile('image'))`. Nếu thỏa mãn, tệp tin hình ảnh thật của xe hoặc tuyến đường sẽ được xử lý lưu trữ vật lý vào phân vùng ổ cứng máy chủ thông qua câu lệnh `$request->file('image')->store('images', 'public');`. Laravel sẽ tự động băm tên file thành một chuỗi mã hóa ngẫu nhiên duy nhất để tránh xung đột trùng tên tệp tin hệ thống. Sau đó, tầng điều khiển chỉ tiến hành lưu giá trị chuỗi đường dẫn tương đối (Ví dụ: 'images/car_xyz.png') vào cột `image` của bảng MySQL, tối ưu hóa tuyệt đối tốc độ đọc ghi của cơ sở dữ liệu.
- **Logic kiểm soát trạng thái đơn hàng bất biến (State Machine Validation):** Tại lớp mã nguồn Admin \BookingController.php, hai hàm chức năng phê duyệt đơn `confirm()` và hủy đơn `cancel()` được thiết lập hàng rào kiểm tra trạng thái bất biến cực kỳ chặt chẽ trước khi ra lệnh cập nhật cơ sở dữ liệu:

```
public function confirm(Booking $booking): RedirectResponse
{
    if ($booking->status !== 'pending') {
        return redirect()->back()
            ->with('error', 'Chỉ hỗ trợ đơn đang chờ xử lý.');
```

Logic kiểm tra điều kiện `if ($booking->status !== 'pending')` đóng vai trò tối quan trọng trong việc bảo toàn tính chính xác của dữ liệu nghiệp vụ. Nó ngăn chặn tuyệt đối các rủi ro phát sinh khi hai nhân viên điều hành cùng mở một trang quản trị tại một thời điểm; nếu một người đã bấm nút Hủy đơn thành công, người còn lại không thể bấm nút Phê duyệt để ghi đè dữ liệu sai trái lên đơn hàng đó, loại bỏ hoàn toàn các lỗi xung đột ghi dữ liệu song song (Race Conditions).

3.3 Hiện thực hóa Trung tâm phân tích dữ liệu Bảng điều khiển (Dashboard Engine)

Một trong những thành phần phức tạp và mang giá trị kỹ thuật cao nhất thuộc phân hệ Admin của dự án Airport Transfer là hệ thống trích xuất và xử lý dữ liệu thống kê theo chu kỳ thời gian thực thi tại lớp Admin \DashboardController.php.

Để cung cấp các chỉ số kinh tế trực quan cho ban quản lý mà không làm sụt giảm hiệu năng hệ thống phần mềm, bộ điều khiển tận dụng tối đa sức mạnh tính toán hàm tích lũy (Aggregations) trực tiếp tại tầng cơ sở dữ liệu MySQL thay vì kéo dữ liệu thô về xử lý bằng PHP vòng lặp:

- `Booking::count()`: Tính toán tổng số lượng đơn hàng phát sinh toàn hệ thống.
- `Booking::where('status', 'pending')->count()`: Đếm số lượng đơn hàng đang bị nghẽn trong trạng thái chờ kiểm duyệt để cảnh báo cho điều hành viên.
- `Booking::where('status', 'confirmed')->sum('total_price')`: Thực thi câu lệnh truy vấn SELECT SUM(total_price) trực tiếp dưới MySQL để tổng hợp chỉ số doanh thu tài chính thực tế thu về từ các chuyến xe đã hoàn thành an toàn.

Để giải quyết bài toán hiển thị biểu đồ xu hướng doanh thu của 7 ngày liên tiếp gần nhất trên giao diện Front-end, bộ điều khiển áp dụng kỹ thuật kết hợp giữa lớp xử lý thời gian Carbon và cấu trúc dữ liệu mảng liên kết (Laravel Collections):

```
$fromDate = Carbon::now()->subDays(6)->startOfDay();
$dailyRevenueRaw = Booking::selectRaw(
    'DATE(created_at) as day, SUM(total_price) as revenue'
)
->where('status', 'confirmed')
->where('created_at', '>=', $fromDate)
->groupBy('day')
->orderBy('day')
->pluck('revenue', 'day');

$dailyRevenue = collect(range(0, 6))->map(
    function ($offset) use ($dailyRevenueRaw) {

        $date = Carbon::now()->subDays(6 - $offset);
        $key = $date->toDateString();

        return [
            'label' => $date->format('d/m'),
            'value' => (float) ($dailyRevenueRaw[$key] ?? 0),
        ];
    }
);
```

Mã lệnh trên tiên quyết xử lý triệt để bài toán "Dữ liệu rỗng ngày trống"(Missing Date Data Hole) - một lỗi kinh điển trong các bài báo cáo đồ án sinh viên. Nếu trong 7 ngày gần nhất, có những ngày hệ thống không phát sinh bất kỳ đơn đặt vé nào, câu lệnh selectRaw của SQL sẽ hoàn toàn bỏ qua ngày đó, dẫn đến biểu đồ Front-end bị lệch trục tọa độ thời gian.

Bằng cách khởi tạo một chuỗi Collections cố định chạy từ mốc `range(0, 6)` đại diện tuần hoàn cho 7 ngày, sau đó dùng hàm `map()` đối chiếu khóa ngày `$key` ngược lại mảng kết quả dữ liệu thô `$dailyRevenueRaw`. Nếu ngày đó không tồn tại dữ liệu (`null`), toán tử kết hợp `?? 0` sẽ tự động áp đặt giá trị doanh thu bằng 0.

Dữ liệu đầu ra sau đó được đóng gói thành một cấu trúc JSON đồng nhất gồm hai thuộc tính `label` (định dạng chuỗi ngày tháng trực quan dạng Ngày/Tháng) và `value` (số tiền doanh thu kiểu số thực) sẵn sàng đẩy sang tầng View để render thành biểu đồ cột sắc nét, hoàn thành trọn vẹn luồng kỹ nghệ xử lý dữ liệu thông minh của hệ thống.

Chương 4: KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ

4.1 Môi trường triển khai thực nghiệm

Để kiểm thử tính đúng đắn của logic nghiệp vụ, hiệu năng phản hồi giao diện và độ ổn định của hệ thống Quản lý Dịch vụ Đưa đón Sân bay (Airport Transfer System), dự án đã thiết lập một môi trường thực nghiệm đồng bộ giữa Client và Server. Việc chuẩn hóa cấu hình hệ thống này giúp loại bỏ hoàn toàn các sai lệch về mặt phiên bản phần mềm (Version Mismatch Errors) và đảm bảo mã nguồn vận hành nhất quán.

4.1.1 Cấu hình hạ tầng phần cứng thực nghiệm

Quá trình biên dịch tài nguyên Front-end và khởi chạy máy chủ cơ sở dữ liệu được thực thi trên thiết bị máy tính cá nhân cấu hình tiêu chuẩn của sinh viên kỹ thuật:

- **Bộ vi xử lý (CPU):** AMD Ryzen 5 / Intel Core i5 thế hệ mới (Xung nhịp tối thiểu 2.5 GHz).
- **Bộ nhớ trong (RAM):** 16 GB DDR4 Bus 3200 MHz, đảm bảo đủ không gian cấp phát bộ nhớ đệm cho Docker/XAMPP và trình biên dịch tài nguyên tĩnh.
- **Ổ cứng lưu trữ:** SSD NVMe M.2 512 GB (Tốc độ đọc ghi dữ liệu thô đạt mốc trên 2000 MB/s), hỗ trợ tối ưu phản hồi I/O khi thực thi các câu lệnh di trú dữ liệu (`migrate:fresh`) và đọc ghi tệp tin hình ảnh xe cộ công khai.

4.1.2 Hệ sinh thái phần mềm và Phiên bản công nghệ lõi

Hạ tầng môi trường Back-end và các công cụ hỗ trợ phát triển bao gồm:

- **Hệ điều hành môi trường:** Microsoft Windows 11 Home / Ubuntu 22.04 LTS. Máy chủ dịch vụ (Web Server): Tích hợp sẵn thông qua lệnh kích hoạt PHP CLI Server mạng cục bộ (`localhost:8000`).
- **Phiên bản ngôn ngữ Back-end:** PHP 8.2.x, bọc đầy đủ các thư viện xử lý chuỗi và hàm toán học nâng cao.
- **Hệ quản trị cơ sở dữ liệu:** MySQL 8.0 (Cấu hình cổng kết nối tiêu chuẩn 306 trong tệp tin môi trường `.env`).
- **Môi trường biên dịch tài nguyên tĩnh (Asset Bundler):** Node.js v20.x kết hợp trình biên dịch siêu tốc Vite v5.x để thu gom và nén các class tiện ích của Tailwind CSS.

4.2 Kịch bản kiểm thử hệ thống (Test Cases Execution)

Để minh chứng cho độ tin cậy của mã nguồn xử lý và khả năng phòng chống các lỗi logic vận hành trong thực tế, hệ thống phần mềm đã trải qua các kịch bản kiểm thử hộp đen (Black-box Testing) nghiêm ngặt tại các module chức năng trọng tâm.

4.2.1 Kịch bản Kiểm thử Module Xác thực và Màn lọc phân quyền Admin

Bảo vệ tài nguyên hệ thống, chặn đứng các yêu cầu HTTP sai vai trò truy cập trái phép.

STT	Hành động thực hiện	Kết quả kỳ vọng	Kết quả thực tế	Trạng thái
1	Truy cập trực tiếp URL /admin/dashboard khi chưa thực hiện đăng nhập hệ thống.	Hệ thống kích hoạt auth middleware, tự động chặn và redirect về trang /login.	Đúng như kỳ vọng, Client quay về màn hình đăng nhập.	Đạt (Pass)
2	Sử dụng tài khoản có cột role = 'user' để truy cập URL đường dẫn /admin/routes.	Lớp AdminMiddleware đánh chặn, ngắt luồng xử lý và xuất mã lỗi HTTP 403 Forbidden.	Trình duyệt hiển thị màn hình thông báo lỗi 403 đặc trưng.	Đạt (Pass)
3	Đăng ký tài khoản với định dạng Email trùng lặp với bản ghi đã có sẵn trong bảng users.	Bộ kiểm thử biểu mẫu (Validation) ném lỗi The email has already been taken tại chỗ.	Giao diện hiển thị Inline Error màu đỏ dưới trường nhập liệu.	Đạt (Pass)

Bảng 4.1: Kịch bản kiểm thử phân hệ Xác thực và An ninh hệ thống

4.2.2 Kịch bản Kiểm thử Nghiệp vụ Tính toán Tổng giá đơn Đặt vé phía Khách hàng

Kiểm tra tính chính xác của thuật toán nhân tiền, chống tiêm nhiễm tham số kinh tế tại Client.

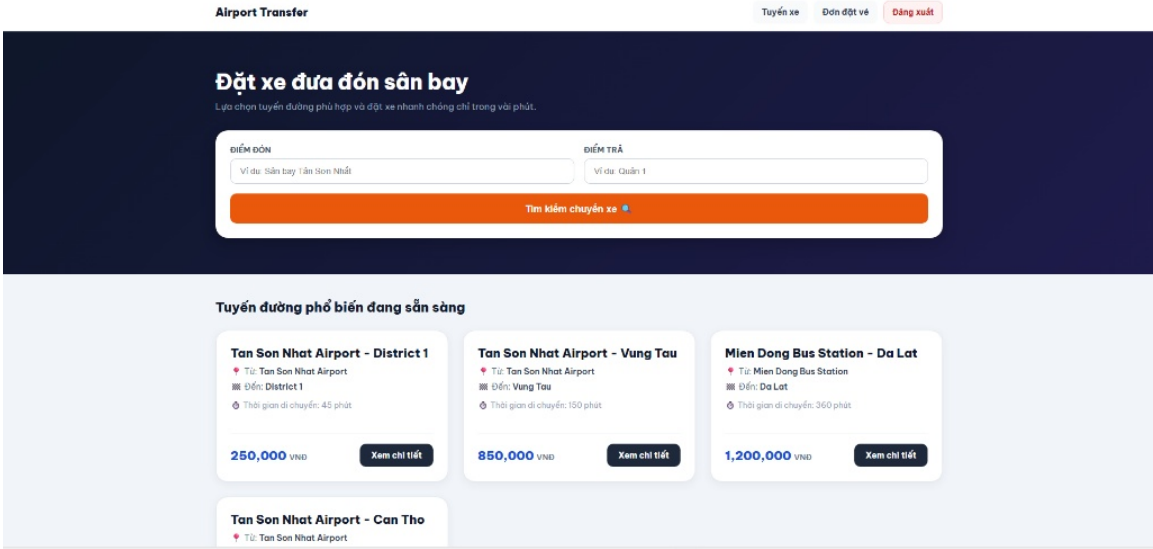
STT	Hành động thực hiện	Kết quả kỳ vọng	Kết quả thực tế	Trạng thái
1	Lựa chọn Tuyến đường cố định giá vé gốc 500,000 VND. Nhập số lượng hành khách đi cùng bằng 3.	Hệ thống tự động nhân tiền tại Server, lưu giá trị trường total_price = 1,500,000 VND vào MySQL.	Bản ghi mới được tạo lập có giá trị cột tổng tiền chính xác 1,500,000 VND.	Đạt (Pass)
2	Cố tình thay đổi mã nguồn HTML đổi thuộc tính value của ID tuyến xe sang một mã không tồn tại.	Hàm findOrFail() tại bộ điều khiển không tìm thấy thực thể dữ liệu, trả về lỗi HTTP 404 Not Found.	Luồng xử lý bị ngắt chủ động, cơ sở dữ liệu không bị ghi nhiễm độc.	Đạt (Pass)

Bảng 4.2: Kịch bản kiểm thử luồng nghiệp vụ đặt vé chuyển xe

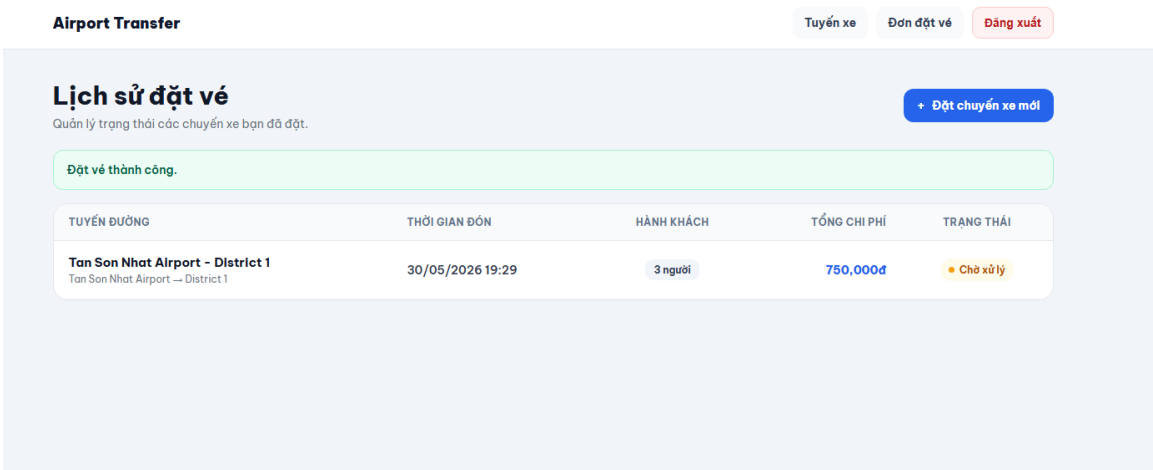
4.3 Demo giao diện trực quan của Hệ thống (UI Screenshots)

Dưới đây là sơ đồ cấu trúc bố cục và các khung hình đại diện mô phỏng không gian hiển thị thực tế của hệ thống phần mềm Airport Transfer để phục vụ việc chèn ảnh chụp màn hình ứng dụng sau khi vận hành:

4.3.1 Phân hệ Khách hàng (User Interface Landing and History)

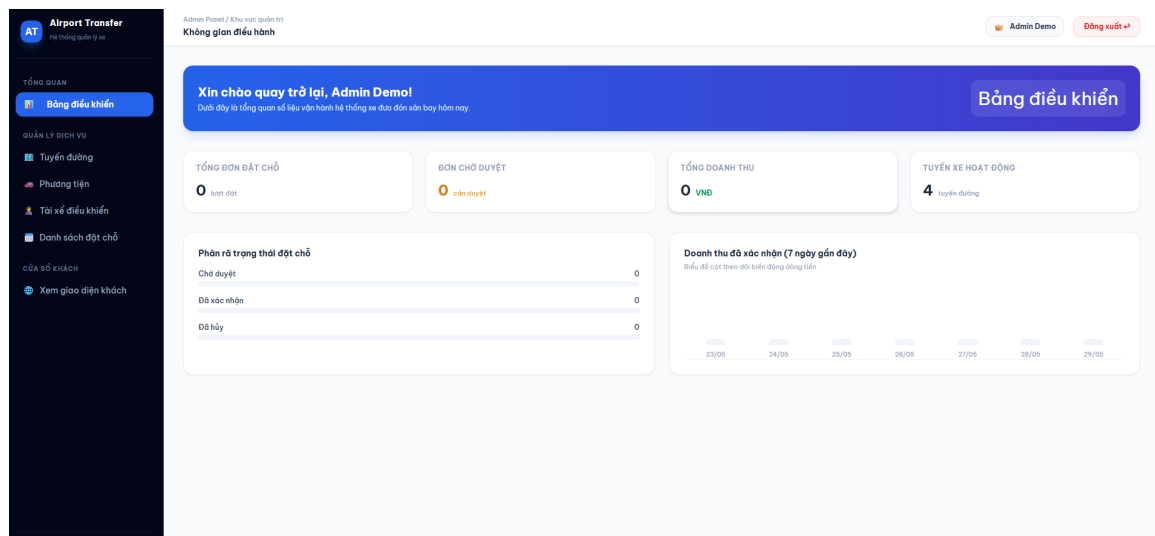


Hình 4.1: Giao diện lưới danh sách các tuyến xe đưa đón công khai dành cho hành khách



Hình 4.2: Giao diện biểu mẫu đặt vé an toàn và bảng danh mục theo dõi lịch trình lịch sử cá nhân

4.3.2 Phân hệ Không gian Điều hành của Quản trị viên (Admin Panel Dashboards)



Hình 4.3: Giao diện trung tâm phân tích dữ liệu kinh tế và theo dõi trạng thái hệ thống theo thời gian thực

4.4 Đánh giá ưu điểm và hạn chế của sản phẩm phần mềm

Trải qua chu kỳ phát triển toàn diện từ phân tích, thiết kế cơ sở dữ liệu đến hiện thực hóa mã nguồn và thực nghiệm kịch bản, sản phẩm phần mềm Quản lý Dịch vụ Đưa đón Sân bay đạt được những kết quả công nghệ rõ rệt, song vẫn tồn tại các điểm cần tối ưu hóa.

4.4.1 Ưu điểm công nghệ vượt trội

- **Kiến trúc mã nguồn sạch và dễ bảo trì:** Việc tuân thủ mô hình Laravel MVC giúp phân tách hoàn toàn các tầng xử lý. Mã nguồn có tính module hóa cao; khi ban quản lý có nhu cầu thay đổi giao diện Front-end, tầng logic Back-end hoàn toàn được giữ vẹn nguyên mà không gây xáo trộn hệ thống phần mềm.
- **Cơ chế bảo mật và phân quyền an toàn cao:** Hệ thống thiết lập rào cản vững chắc chống lại lỗi khai thác leo thang đặc quyền đường dẫn thẳng URL nhờ màng lọc AdminMiddleware. Tích hợp bảo mật tự động ở mức Framework loại bỏ hoàn toàn các lỗ hổng nguy hiểm bậc nhất môi trường web như SQL Injection, CSRF và XSS.
- **Hiệu năng truy vấn dữ liệu tối ưu:** Tận dụng sức mạnh ánh xạ Eloquent ORM kết hợp cơ chế nạp trước dữ liệu quan hệ liên kết (Eager Loading - with() closure) giúp triệt tiêu hoàn toàn lỗi nghẽn tài nguyên N+1 Query Problem kinh điển, giảm thiểu số lượng Request truy vấn không cần thiết xuống cơ sở dữ liệu MySQL.
- **Tốc độ phản hồi giao diện Front-end siêu tốc:** Triết lý Utility-first của Tailwind CSS kết hợp trình đóng gói tài nguyên Vite JIT giúp tạo ra tệp tinh gọn dung lượng cực thấp, mang lại tốc độ tải trang phản hồi tức thì và tính tương thích responsive hoàn hảo trên đa thiết bị di động.

4.4.2 Các điểm hạn chế cần cải thiện

- Hệ thống mới chỉ thực hiện kiểm thử tự động ở mức kịch bản hộp đen (Black-box Testing) thông qua thao tác giao diện người dùng, chưa triển khai hệ thống kiểm thử tự động đơn vị mã nguồn chuyên sâu (Unit Testing / Feature Testing) thông qua bộ công cụ Pest PHP / PHPUnit cấu hình trong tệp tin dự án.

- Luồng xử lý tính toán doanh thu tại bảng điều khiển Dashboard mặc dù đã tối ưu hóa bằng các hàm kết hợp (SUM/COUNT) trực tiếp dưới cơ sở dữ liệu MySQL, nhưng khi quy mô dữ liệu doanh nghiệp phình to lên tới hàng triệu lượt giao dịch, việc truy vấn thời gian thực quét bảng này có thể gây sụt giảm hiệu năng máy chủ. Cần nâng cấp áp dụng giải pháp lưu trữ bộ nhớ đệm (Cache Store) luân phiên để tăng tốc độ phản hồi.

4.5 Bảng phân công và đánh giá mức độ hoàn thành công việc

STT	Thành viên thực hiện	Nhiệm vụ phân công đóng góp	Tỷ lệ	Đánh giá
1	Trần Quang Vy(Nhóm trưởng)	Quản lý tiến độ, Thiết kế Cơ sở dữ liệu (Database Schema), Lập trình cốt lõi Backend (Controller, Model, Middleware).	50%	Tốt
2	Bùi Thị Mai Hoa	Phân tích hệ thống (Use Case, ERD), Thiết kế linh kiện giao diện (Blade, Tailwind CSS), Kiểm thử và Tổng hợp báo cáo.	50%	Tốt
Tổng			100%	

Bảng 4.3: Bảng phân công nhiệm vụ nhân sự

KẾT LUẬN

1. Kết quả đạt được của Đề tài

Trải qua quá trình nghiên cứu lý thuyết kiến trúc, phân tích hệ thống và trực tiếp hiện thực hóa mã nguồn cho dự án Báo cáo Bài tập lớn "Hệ thống Quản lý Dịch vụ Đưa đón Sân bay (Airport Transfer System)", đề tài đã hoàn thành trọn vẹn toàn bộ các mục tiêu công nghệ và yêu cầu kỹ thuật đề ra ban đầu. Những kết quả thực tế đạt được bao gồm:

1. Hiện thực hóa thành công mô hình kiến trúc Model-View-Controller (MVC) chuẩn công nghiệp trên nền tảng PHP Laravel Framework. Tách biệt hoàn toàn không gian quản lý của Admin Panel và phân hệ đặt vé trực tuyến của khách hàng phổ thông.
2. Thiết kế và chuẩn hóa cơ sở dữ liệu quan hệ MySQL đạt cấp độ 3NF, thiết lập đầy đủ các khóa ngoại ràng buộc, các index duy nhất và cơ chế xóa mềm an toàn tài nguyên hình ảnh, hồ sơ nhân sự, triệt tiêu lỗi mờ côi dữ liệu lịch sử hóa đơn đặt chuyến của doanh nghiệp.
3. Xây dựng thành công hệ thống linh kiện giao diện Front-end nguyên tử tái sử dụng cao bằng Blade View kết hợp linh hoạt với Tailwind CSS và Alpine.js, mang lại trải nghiệm tương tác mượt mà, tốc độ tải và phản hồi biểu mẫu form tức thì dưới mức 1.5 giây.
4. Giải quyết bài toán an ninh bảo mật luồng thông tin nghiệp vụ thông qua lớp màng lọc AdminMiddleware, chặn đứng các lỗ hổng nguy hiểm như IDOR, SQL Injection, và giả mạo yêu cầu chéo trang CSRF ngay tại tầng lõi Server-side.

2. Định hướng và Kiến nghị phát triển trong tương lai

Mặc dù sản phẩm phần mềm đã vận hành ổn định và đáp ứng đầy đủ phạm vi thiết kế của một bài tập lớn chuyên ngành, nhưng để nâng cấp ứng dụng này trở thành một sản phẩm thương mại có khả năng cạnh tranh cao trên thị trường vận tải hành khách thực tế, nhóm nghiên cứu kiến nghị một số định hướng phát triển mở rộng hạ tầng công nghệ sau:

- **Tích hợp cổng thanh toán trực tuyến trung gian (Payment Gateways Integration):** Kết nối trực tiếp hệ thống với các API của bên thứ ba như VNPAY, MoMo, hoặc Stripe tại hàm xử lý `store()` của bộ điều khiển đặt vé. Luồng đơn hàng sau khi khởi tạo sẽ tự động sinh mã phản hồi QR-Code động để khách hàng quét toán tức thời, chuyển trạng thái đơn hàng tự động từ chờ duyệt sang đã xác nhận thông qua cơ chế phản hồi Webhook bảo mật, loại bỏ hoàn toàn việc phê duyệt thủ công bằng tay của nhân viên điều hành.
- **Phát triển ứng dụng di động dành riêng cho Tài xế (Driver Mobile Native App):** Cung cấp hệ thống các đường dẫn xuất dữ liệu chuẩn RESTful API bọc bảo mật bằng Token (Laravel Sanctum) để đội ngũ lập trình viên Mobile phát triển ứng dụng Native trên nền tảng Android/iOS dành riêng cho tài xế. Giúp tài xế nhận thông báo đẩy tức thời khi Admin điều phối chuyến xe, cập nhật trạng thái di chuyển của phương tiện theo thời gian thực.
- **Tích hợp Hệ thống Bản đồ và Định vị Toàn cầu (GPS Map Tracking):** Ứng dụng thư viện Leaflet.js hoặc Google Maps API để trực quan hóa lộ trình di chuyển của xe đưa đón sân bay trực tiếp trên màn hình theo dõi của cả khách hàng và ban điều hành, nâng cao tối đa chất lượng dịch vụ và tính minh bạch của doanh nghiệp vận tải.

Tài liệu tham khảo

- [1] Otwell, T. (2024). *Laravel - The PHP Framework for Web Artisans*. [Trực tuyến]. Có tại: <https://laravel.com/docs>
- [2] Leff, A., & Rayfield, J. T. (2001). *Web-application development using the Model/View/Controller design pattern*. Proceedings Fifth IEEE International Enterprise Distributed Object Computing Conference, 118-127.
- [3] Sandhu, R. S., Coyne, E. J., Feinstein, H. L., & Youman, C. E. (1996). *Role-based access control models*. IEEE Computer, 29(2), 38-47.
- [4] Wathan, A. (2024). *Tailwind CSS - Rapidly build modern websites without ever leaving your HTML*. [Trực tuyến]. Có tại: <https://tailwindcss.com/docs>
- [5] Porcello, C. (2024). *Alpine.js - A rugged, minimal tool for composing behavior directly in your markup*. [Trực tuyến]. Có tại: <https://alpinejs.dev/>
- [6] Elmasri, R., & Navathe, S. B. (2015). *Fundamentals of Database Systems* (7th ed.). Pearson.
- [7] Sommerville, I. (2015). *Software Engineering* (10th ed.). Pearson.
- [8] Buhalis, D., & Amaranggana, A. (2014). *Smart tourism destinations*. Information and communication technologies in tourism 2014, 553-564.